

Análise do Impacto de um Pré-Processamento Simétrico $O(n)$ na Redução de Inversões e Eficiência de Algoritmos de Ordenação Adaptativos e Não Adaptativos

Samuel da Penha Nascimento¹, Francisco das Chagas Rocha¹

¹Coordenação do Curso de Sistemas de Computação, Universidade Estadual do Piauí
Campus Prof. Alexandre Alves de Oliveira
Parnaíba, Piauí - Brasil

samuelnascimento@aluno.uespi.br, rocha@phb.uespi.br

Abstract. *Quadratic sorting algorithms remain relevant in resource-constrained systems but are sensitive to inversion counts. This study investigates the impact of inversions on sorting performance and their reduction through preprocessing. The objective is to assess the effectiveness of this technique for adaptive and non-adaptive algorithms. Five algorithms were compared across six input topologies, using vectors of up to 10^6 elements and $O(n)$ preprocessing. The technique reduced inversions by up to 100% at 1.2 ns/element and achieved gains of up to 99.9% in adaptive methods, while $O(n \log n)$ algorithms showed inconsistent results.*

Resumo. *Algoritmos quadráticos permanecem relevantes em sistemas com recursos restritos, mas são sensíveis às inversões. Investiga-se o impacto das inversões no desempenho de algoritmos de ordenação e sua redução por pré-processamento. Objetiva-se avaliar a eficácia dessa técnica em algoritmos adaptativos e não adaptativos. Foram comparados cinco algoritmos em seis topologias, com vetores de até 10^6 elementos, mediante pré-processamento $O(n)$. A técnica reduziu até 100% das inversões a 1,2 ns/elemento e gerou ganhos de até 99,9% nos métodos adaptativos, enquanto os $O(n \log n)$ apresentaram resultados inconsistentes.*

1. Introdução

A ordenação de dados é uma operação fundamental na Ciência da Computação, sendo essencial em sistemas de busca, bancos de dados e aplicações em tempo real [Knuth 1973, Sedgewick and Wayne 2011]. Embora métodos com complexidade assintótica $O(n \log n)$, como Merge Sort e Quicksort, sejam preferíveis para o tratamento de grandes volumes de dados, algoritmos de natureza quadrática $O(n^2)$, como Insertion Sort, Selection Sort e Bubble Sort, permanecem relevantes em cenários específicos [Cormen et al. 2022]. Essa permanência está relacionada à simplicidade estrutural, ao baixo consumo de memória por operarem *in-place* e à eficiência prática em conjuntos de dados pequenos ou parcialmente ordenados [Cormen et al. 2022, Ziviani 2021].

A eficiência prática de um algoritmo de ordenação, entretanto, não depende exclusivamente de sua complexidade assintótica, sendo também influenciada pelas propriedades da sequência de entrada [Estivill-Castro and Wood 1992]. Nesse contexto, a inversão, definida como um par de elementos que satisfaz $A[i] > A[j]$ para $i < j$, constitui uma medida quantitativa do grau de desordem de um arranjo [Knuth 1973, Mannila 1985]. O número de inversões permite caracterizar diferentes níveis de ordenação prévia (*presortedness*), uma vez que sequências com maior quantidade de inversões apresentam maior desordem em relação à ordem final [Mannila 1985]. Essa característica pode influenciar significativamente o custo de execução de determinados algoritmos, especialmente daqueles cuja quantidade de operações depende do grau de desordem da entrada [Estivill-Castro and Wood 1992].

Apesar dessa relação entre desordem da entrada e custo de execução, algoritmos quadráticos podem apresentar baixa capacidade de adaptação às condições iniciais dos dados [Mannila 1985, Estivill-Castro and Wood 1992]. Em particular, entradas com elevada quantidade de inversões podem aumentar o número de movimentações necessárias durante a ordenação, contribuindo para a degradação do desempenho de métodos como o Insertion Sort [Cormen et al. 2022]. Nesse cenário, uma etapa preliminar capaz de reduzir a desordem da entrada pode diminuir o trabalho realizado posteriormente pelo algoritmo de ordenação. Surge, assim, a seguinte questão de pesquisa: em que medida a implementação de uma etapa de pré-processamento voltada à redução de inversões impacta o desempenho prático de algoritmos de ordenação quadráticos e quasilineares?

Formalmente, denota-se por C_{original} o custo computacional temporal da ordenação direta da entrada. Ao introduzir o pré-processamento simétrico com custo C_{pre} , o algoritmo de ordenação subsequente passa a atuar sobre um vetor modificado, despendendo um tempo reduzido C_{sort} . O objetivo deste trabalho é propor, implementar e avaliar um algoritmo simétrico de pré-processamento de baixo custo $O(n)$, projetado para reduzir o número de inversões de um vetor de entrada. Busca-se verificar experimentalmente se a abordagem híbrida apresenta maior eficiência temporal que a aplicação direta dos algoritmos convencionais, identificando rigorosamente as condições teóricas e práticas nas quais a condição $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$ é satisfeita.

A relevância deste estudo está na investigação de uma estratégia capaz de melhorar o comportamento prático de algoritmos de ordenação sem modificar sua estrutura fundamental ou introduzir elevado custo computacional adicional. A redução do grau de desordem antes da ordenação principal pode diminuir o número de operações realizadas em entradas desfavoráveis, especialmente em métodos sensíveis à quantidade de inversões [Estivill-Castro and Wood 1992, Hwang et al. 2000]. Além disso, o estudo contribui para a análise de algoritmos adaptativos ao investigar as condições nas quais o custo do pré-processamento é compensado pela redução do trabalho realizado durante a ordenação [Mannila 1985, Hwang et al. 2000].

A pesquisa adota uma abordagem quantitativa, aplicada e experimental [Gil 2019, Sampieri et al. 2021]. O estudo compreende a modelagem formal do algoritmo de pré-processamento e sua implementação em Rust. A avaliação experimental foi realizada por meio de *benchmarks* automatizados com a biblioteca Criterion, utilizando vetores sintéticos com tamanhos variando de 1.000 a 1.000.000 elementos e

níveis controlados de inversão, abrangendo situações de pior caso, ordenação parcial e topologias estruturadas adversariais inspiradas em padrões clássicos de teste de ordenação. Os experimentos compararam o tempo de execução da abordagem híbrida com o dos algoritmos de ordenação aplicados diretamente às mesmas entradas.

Este artigo está estruturado em quatro seções, além desta introdução. A Seção 2 apresenta a Fundamentação Teórica. A Seção 3 descreve a Metodologia, a formalização do algoritmo de pré-processamento proposto, dos algoritmos de ordenação avaliados e os procedimentos experimentais. A Seção 4 apresenta e discute os resultados obtidos. Por fim, a Seção 5 apresenta as Considerações Finais.

2. Fundamentação Teórica

Nesta seção, são apresentados os fundamentos teóricos necessários à compreensão da pesquisa, abrangendo o conceito de inversões como medida de desordem, os princípios da ordenação adaptativa e as características dos algoritmos avaliados, contemplando métodos quadráticos e quasilineares. Em seguida, são discutidos o conceito de pré-processamento, os princípios do Pré-processamento Simétrico e sua finalidade na redução de inversões, além dos trabalhos relacionados que fundamentam teoricamente a abordagem adotada e situam a contribuição desta pesquisa no contexto da literatura. Esses fundamentos estabelecem as bases para compreender a relação entre a estrutura da entrada, o comportamento dos algoritmos de ordenação e os efeitos esperados da aplicação do pré-processamento proposto.

2.1. Inversões e Ordenação Adaptativa

A eficiência prática dos algoritmos de ordenação tradicionais frequentemente depende da disposição inicial dos elementos no conjunto de dados de entrada [Cormen et al. 2022, Estivill-Castro and Wood 1992]. Para formalizar e quantificar o grau de desordem de uma sequência, utiliza-se classicamente o conceito de *inversão* [Knuth 1973]. Dado um vetor a composto por n elementos, uma inversão é definida formalmente como um par de índices (i, j) tal que $i < j$ e $a[i] > a[j]$ [Knuth 1973, Sedgewick and Wayne 2011]. O número total de inversões I constitui uma medida do grau de desordem da entrada e está diretamente relacionado ao esforço necessário para ordená-la por meio de operações locais [Mannila 1985]. A Figura 1 ilustra visualmente a identificação de inversões em um vetor arbitrário.

A relação entre o volume de inversões I e a complexidade de execução dá origem à propriedade de *adaptabilidade* em algoritmos de ordenação [Estivill-Castro and Wood 1992]. Um algoritmo é considerado adaptativo quando sua complexidade temporal melhora sensivelmente na presença de um ordenamento prévio parcial na entrada [Mannila 1985]. Um exemplo clássico é o *Insertion Sort*, cuja execução consome $O(n + I)$ operações. Em um vetor estritamente ordenado ($I = 0$), o algoritmo atinge tempo linear $O(n)$, enquanto, no pior caso ($I = O(n^2)$), degrada para tempo quadrático [Cormen et al. 2022, Knuth 1973].

2.2. Algoritmos Avaliados

Para avaliar o impacto do pré-processamento simétrico sobre diferentes paradigmas de ordenação, selecionou-se um conjunto representativo de algoritmos com características

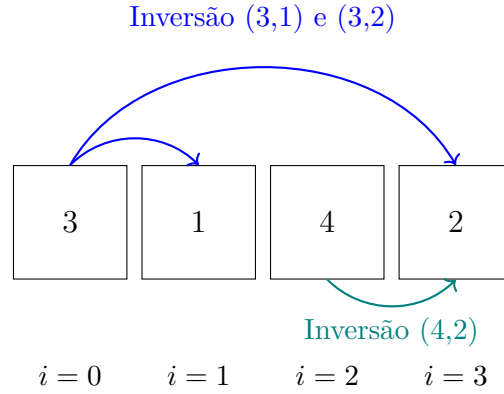


Figura 1. Representação visual do conceito de inversão em um vetor $a = [3, 1, 4, 2]$. O número total de inversões é $I = 3$, correspondente aos pares ordenados onde elementos maiores antecedem elementos menores em posições anteriores.

distintas quanto à complexidade temporal, adaptabilidade e sensibilidade à ordem inicial dos elementos. A seleção foi realizada de modo a contemplar tanto algoritmos cuja eficiência pode ser diretamente influenciada pela quantidade de desordem presente na entrada quanto métodos cujo comportamento é menos dependente dessa propriedade.

No grupo de algoritmos com complexidade temporal quadrático $O(n^2)$ no pior caso, foram avaliados o *Bubble Sort*, o *Insertion Sort* e o *Selection Sort* [Cormen et al. 2022]. Os dois primeiros apresentam comportamento adaptativo, reduzindo o número de operações conforme o grau de ordenamento prévio da entrada [Estivill-Castro and Wood 1992], o que os torna relevantes para analisar os impactos da redução de inversões. Em contrapartida, o *Selection Sort* é não adaptativo e mantém um número fixo de comparações independentemente da configuração inicial [Estivill-Castro and Wood 1992], servindo como contraponto aos algoritmos sensíveis ao pré-processamento.

No grupo dos algoritmos de complexidade quasilinear, foram avaliados o *Merge Sort*, cuja complexidade temporal é $O(n \log n)$, e o *Quicksort*, cujo desempenho médio é $O(n \log n)$, embora seu pior caso assintótico seja $O(n^2)$ [Cormen et al. 2022]. A inclusão desses métodos permite verificar se os efeitos observados nos algoritmos quadráticos permanecem relevantes quando o custo assintótico da ordenação é inferior. Dessa forma, o experimento não se restringe a algoritmos diretamente relacionados à contagem de inversões, permitindo analisar também a influência do pré-processamento em métodos cuja estratégia de ordenação apresenta características distintas.

Assim, o conjunto avaliado contempla três dimensões relevantes para a análise experimental: algoritmos quadráticos adaptativos (*Bubble Sort* e *Insertion Sort*), um algoritmo quadrático não adaptativo (*Selection Sort*) e algoritmos quasilineares baseados em estratégias distintas de ordenação (*Merge Sort* e *Quicksort*). Essa composição permite verificar não apenas se o pré-processamento simétrico reduz a quantidade de inversões da entrada, mas também em que medida essa redução se converte em ganhos de desempenho para algoritmos com diferentes graus de dependência em relação à ordem inicial dos dados.

2.3. Pré-Processamento Simétrico

O conceito de pré-processamento, no contexto do projeto e análise de algoritmos, refere-se a uma etapa preliminar de transformação ou reestruturação dos dados de entrada realizada antes da execução do algoritmo principal [Cormen et al. 2022]. O propósito dessa fase preparatória é alterar a disposição ou as propriedades da instância do problema, tornando a resolução subsequente mais simples ou computacionalmente mais eficiente. De forma conceitual e intuitiva, o pré-processamento atua de maneira análoga à organização prévia de um ambiente ou material de trabalho antes da execução de uma tarefa complexa: embora a etapa de preparação exija um custo computacional inicial, a redução no esforço exigido no processamento principal compensa esse investimento prévio.

Para que uma etapa de pré-processamento seja viável em problemas de ordenação, seu custo temporal deve ser estritamente dominado pela complexidade do algoritmo de ordenação que o sucede [Cormen et al. 2022, Sedgewick and Wayne 2011]. Dessa forma, um pré-processamento ideal deve apresentar complexidade temporal linear $O(n)$, garantindo que o *overhead* introduzido seja assintoticamente desprezível em relação ao custo total da ordenação — seja em algoritmos quasilineares $O(n \log n)$ no caso médio ou quadráticos $O(n^2)$ no pior caso [Knuth 1973].

Especificamente nesta pesquisa, a etapa de pré-processamento é denominada *simétrica* por aplicar uma varredura bidirecional e espelhada a partir das extremidades do vetor. Essa reestruturação visa reduzir o número total de inversões I da entrada ao mover elementos discrepantes para posições mais próximas de seus destinos finais em um único passe de complexidade $O(n)$. Como resultado, ao fornecer uma estrutura parcialmente reordenada para os algoritmos subsequentes, busca-se mitigar o número de comparações e trocas operadas tanto por métodos adaptativos quanto por métodos não adaptativos.

2.4. Trabalhos Relacionados

A literatura apresenta diferentes abordagens para caracterizar o grau de ordenação de uma sequência e explorar essa informação no processo de ordenação. Mannila [Mannila 1985] teve como objetivo estudar o conceito de pré-ordenação e estabelecer medidas capazes de quantificar diferentes formas de desordem em uma sequência. Para isso, o autor definiu propriedades gerais para medidas de pré-ordenação e introduziu o conceito de algoritmo de ordenação ótimo em relação a uma determinada medida. Entre as medidas consideradas encontra-se o número de inversões, representado por $Inv(X)$, que quantifica pares de elementos fora de ordem. Como resultado, o trabalho apresentou exemplos de algoritmos ótimos para diferentes medidas e demonstrou que o *Insertion Sort* é ótimo em relação a três medidas naturais de pré-ordenação, estabelecendo fundamentos teóricos para o estudo de algoritmos adaptativos. O trabalho foi apresentado como resumo estendido no ICALP em 1984 e publicado em versão journal no *IEEE Transactions on Computers* em 1985 [Mannila 1985]; a referência aqui adotada corresponde à versão journal de 1985 (vol. C-34, pp. 318–325).

Posteriormente, Estivill-Castro e Wood [Estivill-Castro and Wood 1992] realizaram um levantamento sistemático sobre algoritmos de ordenação adaptativos, considerando o grau de desordem existente na sequência de entrada. O trabalho teve

como objetivo apresentar os principais conceitos, medidas de desordem e resultados então estabelecidos sobre *adaptive sorting*. A metodologia consistiu em uma revisão e síntese dos resultados teóricos e práticos disponíveis, incluindo relações entre medidas de desordem e algoritmos capazes de explorar a ordenação prévia dos dados. Entre os resultados discutidos, os autores destacam que diversos algoritmos podem apresentar complexidade dependente tanto do tamanho da entrada quanto de sua desordem. O survey também registra que determinados algoritmos adaptativos apresentam desempenho competitivo em sequências aleatórias e ganhos significativos em entradas quase ordenadas, consolidando a adaptatividade como uma abordagem relevante para explorar a ordem existente nos dados.

Em uma abordagem diretamente relacionada ao uso de transformações preliminares, Hwang, Yang e Yeh [Hwang et al. 2000] investigaram algoritmos de *presorting* sob uma perspectiva de caso médio, utilizando a redução no número de inversões como medida de desempenho. Os autores introduziram uma formulação para quantificar o efeito dessas etapas preliminares por meio da redução média de inversões produzida durante o processamento. A análise considerou etapas de algoritmos conhecidos, incluindo *Shellsort* e *Quicksort*, avaliando a variação da estatística de inversões após sua execução. Como resultados, foram obtidos valores para a esperança e a variância da redução no número de inversões, além de funções geradoras associadas a essa redução. O estudo também investigou a possibilidade de empregar etapas de *presorting* antes da ordenação propriamente dita, fornecendo uma formulação analítica para avaliar esse tipo de estratégia. O arcabouço de Hwang et al. permitiria, em princípio, derivar a redução esperada do pré-processamento simétrico sob entrada aleatória uniforme: sob permutação aleatória, $\Pr[A[i] > A[n-1-i]] = 0,5$, de modo que o número esperado de trocas simétricas é $n/4$; cada troca elimina pelo menos a inversão do par $(i, n-1-i)$ e, em média, afeta $O(n)$ outras relações, o que sugere ganho linear mas cuja quantificação exata exige a função geradora do artigo. Uma derivação fechada rigorosa nos moldes de Hwang et al. é deixada como trabalho futuro; empiricamente, a redução média observada em entrada aleatória foi de $\sim 45\%$ (Tabela 3), compatível com a ordem de grandeza prevista por esse modelo.

A literatura mais recente reforça a relevância prática desses efeitos microarquiteturais. Edelkamp e Weiss [Edelkamp and Weiß 2016] demonstraram que erros de predição de desvios (*branch mispredictions*) podem dominar o custo do Quicksort e propuseram o *BlockQuicksort* para mitigá-los, fornecendo base para a hipótese de degradação observada no *Selection Sort* e no Quicksort deste trabalho. Auger et al. [Auger et al. 2018] analisaram a complexidade de pior caso do *Timsort*, algoritmo adaptativo amplamente adotado, evidenciando que a exploração de corridas (*runs*) pré-existentes permanece tema ativo de engenharia de ordenação. LaMarca e Ladner [LaMarca and Ladner 1999] já haviam mostrado a influência de *caches* no desempenho de ordenação, aspecto também discutido por Kaligosi e Sanders [Kaligosi and Sanders 2006] ao quantificar o impacto de *branch mispredictions* em algoritmos de ordenação. Esses trabalhos posicionam a presente técnica de baixo custo como instância de heurística de pré-processamento cujo trade-off deve ser avaliado conjuntamente nos níveis lógico (inversões) e microarquitetural.

Diferenciando-se das abordagens mapeadas, a presente pesquisa avalia um

algoritmo de Pré-processamento Simétrico que atua na mitigação direta de inversões em tempo linear, preenchendo a lacuna de otimização de baixo custo para algoritmos de ordenação.

3. Metodologia

Esta seção detalha a abordagem metodológica adotada para a realização deste trabalho, descrevendo a caracterização da pesquisa, a configuração do ambiente experimental e os procedimentos empregados no benchmarking dos algoritmos. A investigação foi conduzida por meio de um protocolo experimental controlado, utilizando execuções compiladas em linguagem Rust e métricas estatísticas rigorosas fornecidas pela biblioteca *Criterion*. Para assegurar a abrangência dos testes e a replicabilidade dos resultados, foram projetadas seis topologias distintas de vetores sob controle de semente aleatória. A escolha desta metodologia visa garantir a precisão temporal e a validade interna da avaliação do pré-processamento simétrico, fornecendo um caminho metodológico transparente e totalmente reproduzível.

3.1. Tipo de Pesquisa e Configuração

O presente estudo caracteriza-se como uma pesquisa aplicada, quantitativa, explicativa e experimental [Gil 2019, Sampieri et al. 2021]. A caracterização apoia-se no isolamento e na manipulação controlada da topologia das entradas para medir a variação no tempo de execução e na contagem de inversões.

Os experimentos foram realizados em ambiente controlado, cujas especificações de hardware, sistema operacional e ferramentas são apresentadas na Tabela 1. Para reduzir possíveis fontes de variabilidade, o plano de energia foi mantido em alto desempenho e os processos de segundo plano foram minimizados durante a execução dos testes. O código foi compilado em modo *release*, com nível máximo de otimização (`opt-level = 3`). A geração das entradas empregou uma semente fixa, garantindo reprodutibilidade e correspondência entre os conjuntos submetidos às diferentes condições experimentais.

Tabela 1. Ambiente experimental e ferramentas utilizadas.

Componente	Configuração / Especificação
Sistema operacional	Windows 11 Pro (build 10.0.26200)
Processador	AMD Ryzen 7 8700G (8 núcleos / 16 threads)
Memória	16 GB DDR5 5600MT/s (dual channel)
Linguagem	Rust 1.96.0 (perfil <i>release</i>)
Ferramentas de Medição	Criterion 0.5 e Ferramenta CLI do Projeto
Geração de dados	Seed 42 (<i>ChaCha8Rng</i>), entradas pareadas

Para avaliar a resiliência e a adaptabilidade dos algoritmos sob diferentes graus de entropia, foram projetados seis padrões estruturais de vetores de entrada (Tabela 2). A aleatoriedade determinística foi assegurada pelo pareamento rigoroso das sementes de geração entre as abordagens: para cada combinação de topologia e tamanho, um conjunto fixo de 50 vetores é gerado com a semente $42 \oplus n \oplus \text{tipo}$ (via *ChaCha8Rng*) e compartilhado por todos os algoritmos e pelas duas variantes

(pura e com pré-processamento), garantindo a comparabilidade pareada das medições. Vetores determinísticos (Invertido, Zigue-zague e Tartarugas) são idênticos entre as 50 réplicas do *pool*; vetores estocásticos (Aleatório, Duplicados e Quase ordenado) variam entre réplicas pela semente, mas são idênticos entre algoritmos para a mesma réplica.

Os algoritmos quasilineares (*Merge Sort* e *Quicksort*) foram medidos nas instâncias de tamanho $n \in \{1.000, 5.000, 10.000, 100.000, 1.000.000\}$. Para os algoritmos quadráticos (*Insertion Sort*, *Bubble Sort* e *Selection Sort*), as medições foram restritas a $n \leq 100.000$: em $n = 10^6$, cada execução desses métodos demandaria tempo da ordem de minutos, inviabilizando a coleta sistemática de um número estatisticamente significativo de amostras. Essa restrição é documentada e não compromete a comparação, pois a ordem de grandeza dos ganhos desses métodos já se estabiliza em $n = 100.000$.

O custo isolado do pré-processamento (C_{pre}) foi adicionalmente caracterizado em $n \in \{1.000, 5.000, 10.000, 20.000, 100.000, 1.000.000\}$ (Tabela 5 e Figura 3) para melhor evidenciar a linearidade; o ponto $n=20.000$ foi coletado exclusivamente para essa curva de custo via ferramenta CLI (`seed 42`), sob mesmo protocolo de geração pareada, e não faz parte da matriz principal de comparação de tempos de ordenação.

Tabela 2. Conjuntos de dados e caracterização das topologias de entrada.

Topologia	Descrição do Padrão Estrutural
Aleatório	Permutação uniforme de $[0, n-1]$ via <code>shuffle</code> com <code>ChaCha8Rng</code>
Tartarugas	metade superior elevada e crescente; metade inferior baixa e cíclica
Zigue-zague	alternância elemento a elemento; determinístico
Quase ordenado	$[0, n-1]$ ordenado com $n/100$ trocas de pares uniformes (i, j) , $i \neq j$
Duplicados	$a[i] \sim \mathcal{U}\{0, 1, 2\}$ i.i.d. (sem agrupamento espacial intencional)
Invertido	ordem estritamente decrescente; pior caso determinístico

3.2. Especificação e Implementação dos Algoritmos Avaliados

Para assegurar a validade interna dos experimentos, a estabilidade das medições de tempo e a reprodutibilidade dos resultados, todos os algoritmos de ordenação foram implementados na linguagem Rust (versão 1.96.0), operando diretamente sobre fatias mutáveis de inteiros de 32 bits (`&mut [i32]`). As implementações evitaram dependências externas de abstrações de alto nível, padronizando a manipulação de memória e adotando otimizações idiomáticas no nível de instrução.

3.2.1. Algoritmos Quadráticos

- **Bubble Sort:** Instanciado em sua versão adaptativa otimizada. O algoritmo realiza varreduras sucessivas reduzindo o limite superior a cada iteração $(n - 1 - i)$. A adaptabilidade é garantida por uma flag booleana de controle (`trocou`) que monitora a ocorrência de trocas em cada passada; caso uma varredura seja concluída sem nenhuma permuta de elementos, a execução é interrompida precocemente, garantindo complexidade linear $O(n)$ em vetores previamente ordenados.

- **Insertion Sort:** Implementado conforme a mecânica tradicional de inserção direta por deslocamento. Para cada elemento da sub-fatia não ordenada, armazena-se o valor corrente em uma variável temporária (`valor_atual`) e promovem-se deslocamentos à direita dos elementos antecedentes estritamente maiores. A busca e o deslocamento encerram-se assim que a posição correta é encontrada ou ao atingir o limite esquerdo ($j = 0$), momento em que o valor temporário é atribuído. Essa abordagem minimiza o *overhead* de escrita em relação às implementações baseadas em trocas sucessivas (*swaps*).
- **Selection Sort:** Implementado em sua forma clássica não adaptativa e *in-place*. Em cada iteração i do laço externo, o algoritmo varre o segmento não ordenado $[i + 1, n - 1]$ para identificar o índice do menor elemento (`indice_minimo`). A troca (*swap*) com a posição i é executada condicionalmente apenas se o menor elemento encontrado residir em um índice distinto (`indice_minimo  $\neq$  i`). O algoritmo mantém um custo de comparações estrito de $O(n^2)$, independentemente da ordenação prévia da entrada.

3.2.2. Algoritmos Quasilineares

- **Merge Sort:** Desenvolvido segundo a estratégia **top-down** (divisão e conquista) com garantia de ordenação estável. Para eliminar o **overhead** de alocação dinâmica contínua na pilha recursiva — o que distorceria as medições temporais —, a implementação aloca um único vetor auxiliar temporário de tamanho n (`vec[0; len]`) no ponto de entrada da função. Esse **buffer** é passado por referência mutável através das chamadas recursivas. Os pontos médios das sub-fatias são calculados por meio do deslocamento de bits à direita ($(\text{fim} - \text{inicio}) \gg 1$), evitando operações de divisão inteira. A fase de intercalação (*intercalar*) copia os dados da fatia atual para o **buffer** auxiliar e reconstrói o segmento ordenado no vetor original operando com três ponteiros de índice.
- **Quicksort:** Implementado de forma recursiva utilizando particionamento de três vias (*Dutch National Flag* / algoritmo de Dijkstra) aliado à seleção de pivô por mediana de três. O pivô é escolhido como a mediana entre os elementos nos índices inicial, central e final do subvetor. O particionamento de três vias rearranja o subvetor em três regiões contíguas: elementos menores que o pivô, elementos iguais ao pivô e elementos maiores que o pivô, mantendo índices `lt`, `i` e `gt` para delimitar as fronteiras. Essa estratégia garante complexidade $O(n \log n)$ mesmo em presença de muitas chaves duplicadas, evitando a degradação quadrática do particionamento de Lomuto. A recursão é aplicada apenas às regiões de elementos menores (`[inicio, lt-1]`) e maiores (`[gt+1, fim]`) que o pivô.

Em todas as implementações, verificações preliminares de contorno (*early returns*) asseguram que fatias com tamanho $n < 2$ retornem imediatamente sem incorrer em chamadas de função ou alocações desnecessárias.

3.3. Protocolo Experimental e Raciocínio de Cálculo de Inversões

O protocolo de medição empírica foi desenhado para isolar interferências do ambiente e garantir a reprodutibilidade estatística das métricas temporais. A quantificação do tempo de execução utilizou o arcabouço *Criterion.rs*, que executa ciclos automatizados de aquecimento de memória cache (*warm-up*) e realiza reamostragens estatísticas (*bootstrapping*) para mitigar o impacto de interrupções de hardware. Para cada combinação de algoritmo, topologia e tamanho de vetor n , foram calculados o tempo médio, a mediana e os intervalos de confiança de 95% (`confidence_level=0.95`, `significance_level=0.05`). A configuração de medição foi ajustada por perfil: 50 amostras com aquecimento de 2s e janela de medição de 5s para a maioria dos cenários; para os algoritmos quadráticos em $n = 100.000$ (cada iteração dura frações de segundo), 30 amostras com aquecimento de 1s e janela de 5s; e, para os métodos quasilineares em $n = 10^6$, 50 amostras com aquecimento de 2s e janela de 10s. A medição temporal das abordagens submetidas ao pré-processamento contabilizou rigorosamente o custo composto $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$, garantindo a aferição transparente da condição de viabilidade prática $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$.

Critério estatístico: considerou-se uma variação relevante quando os intervalos de confiança de 95% das duas condições (pura e com pré-processamento) não se sobrepõem. Quando os ICs se sobrepõem, a variação foi tratada como não significativa, independentemente da diferença percentual pontual. Esse critério foi aplicado uniformemente em toda a Seção 4 para qualificar expressões como “ganho consistente”, “degradação sistemática” e “resultados mistos”.

Além das métricas temporais, a corretude de cada cenário foi verificada fora da região cronometrada: para todo (algoritmo, topologia, tamanho), confirma-se que a saída é uma permutação ordenada da entrada (comparação com `sort_unstable`) tanto na variante pura quanto na com pré-processamento. O número de inversões de cada vetor do conjunto compartilhado, antes e depois do pré-processamento, é registrado em arquivos auxiliares (*companion*) do harness (`target/criterion/.../_companion/inversoes_{tipo}_{n}.csv`), garantindo que as tabelas de inversões deste artigo derivam exatamente dos mesmos vetores cronometrados.

A quantificação experimental do número exato de inversões I utiliza uma implementação baseada no algoritmo MergeSort modificado, com complexidade temporal $O(n \log n)$ [Cormen et al. 2022]. Durante a etapa de intercalação de dois subvetores ordenados L e R , quando um elemento $R[j]$ é menor que $L[i]$, adiciona-se exatamente $|L| - i$ ao contador de inversões. Essa abordagem elimina o gargalo quadrático da contagem direta $O(n^2)$, permitindo a avaliação de vetores com ordens de grandeza de 10^5 a 10^6 elementos dentro do protocolo experimental.

3.4. Formalização do Pré-Processamento Simétrico

O procedimento de pré-processamento simétrico proposto opera como uma etapa determinística de varredura única sobre o vetor de entrada A de tamanho n . O objetivo desta etapa é reestruturar a disposição dos elementos através de trocas espelhadas e ajustes locais, antecedendo a execução dos algoritmos de ordenação avaliados. A estrutura procedural e as regras de controle do algoritmo estão formalizadas no

Algoritmo 1.

Algoritmo 1: Algoritmo de Pré-Processamento Simétrico

Input: Vetor A de tamanho n
Output: Vetor A parcialmente reorganizado
 $n \leftarrow$ tamanho de A ;
if $n < 2$ **then**
 return;
end
 $ultimo \leftarrow n - 1$;
 $meio \leftarrow \lfloor n/2 \rfloor$;
for $i \leftarrow 0$ **to** $meio - 1$ **do**
 $j \leftarrow ultimo - i$;
 if $A[i] > A[j]$ **then**
 trocar($A[i]$, $A[j]$);
 end
 if $i + 1 < meio$ **then**
 if $A[i] > A[i + 1]$ **then**
 trocar($A[i]$, $A[i + 1]$);
 end
 end
 if $j - 1 > meio$ **then**
 if $A[j] < A[j - 1]$ **then**
 trocar($A[j]$, $A[j - 1]$);
 end
 end
end

O algoritmo itera sobre a primeira metade do vetor, varrendo os índices $i \in [0, \lfloor n/2 \rfloor - 1]$. A cada iteração, calcula-se o índice simétrico correspondente no extremo oposto do vetor, dado por $j = (n - 1) - i$. A execução do laço é composta por três verificações condicionais estruturadas:

1. **Troca Simétrica Extrema:** Avalia-se a relação de ordem entre as extremidades opostas. Se $A[i] > A[j]$, efetua-se a troca direta dos elementos, posicionando o menor elemento no segmento esquerdo e o maior no segmento direito.
2. **Ajuste Adjacente Esquerdo:** Para os índices em que $(i + 1) < \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade esquerda. Se $A[i] > A[i + 1]$, realiza-se a troca entre os elementos adjacentes.
3. **Ajuste Adjacente Direito:** Para os índices em que $(j - 1) > \lfloor n/2 \rfloor$, verifica-se a ordenação local na metade direita. Se $A[j] < A[j - 1]$, executa-se a troca entre os elementos adjacentes.

As condições de contorno garantem que as trocas adjacentes operem estritamente dentro de seus respectivos subdomínios (esquerdo e direito), sem ultrapassar o ponto médio do vetor. Em termos de complexidade computacional, o algoritmo executa exatamente $\lfloor n/2 \rfloor$ iterações. Em cada passo do laço, realiza-se um número

constante de comparações (no máximo 3) e trocas (no máximo 3), resultando em uma complexidade temporal de pior caso $T(n) = O(n)$. Como a reestruturação dos dados é realizada *in-place* no próprio vetor A , a complexidade espacial auxiliar é estritamente $O(1)$.

3.5. Ameaças à Validade

As medições de tempo físico foram protegidas contra ruídos por meio do aquecimento prévio da CPU e da alternância da ordem de execução. A ausência de estatísticas de baixo nível de hardware, como *cache misses*, decorre da indisponibilidade da interface `perf_event_open`, específica do Linux, no ambiente Windows utilizado. Todos os experimentos podem ser reproduzidos por meio do comando `cargo bench` e das rotinas disponibilizadas no repositório do projeto [Nascimento 2026].

4. Resultados e Discussão

Esta seção apresenta a análise e discussão dos resultados experimentais obtidos, avaliando o impacto do pré-processamento simétrico sobre a estrutura dos dados e o tempo de execução dos algoritmos. A discussão segue uma estrutura analítica que parte do impacto direto na redução de inversões e da interpretação do viés geométrico observado no cenário invertido, avança para a mensuração do tempo total de ordenação em algoritmos quadráticos e quasilineares, e culmina na interpretação dos efeitos microarquiteturais, como a localidade de referência e o comportamento das caches de CPU. O objetivo é demonstrar de forma consolidada os cenários em que a técnica oferece ganhos reais de desempenho e entender as limitações físicas e algorítmicas envolvidas.

4.1. Redução de Inversões e Viés Estrutural das Entradas

A Figura 2 ilustra a variação das inversões observadas nos vetores de 10.000 elementos, cujos valores quantitativos exatos e porcentagens de redução estão detalhados na Tabela 3. Observa-se que o pré-processamento é capaz de mitigar expressivamente a desordem em múltiplos cenários. Contudo, a interpretação da eliminação total das inversões (100%) no cenário *Invertido* exige sobriedade teórica e analítica.

Tabela 3. Redução de inversões proporcionada pelo pré-processamento simétrico em vetores de 10.000 elementos (média dos 50 vetores do *pool* pareado; mesma semente dos tempos da Tabela 4).

Tipo	Inversões	Pós-pré	Redução
Aleatório	24.960.120	13.681.366	45,19%
Tartarugas	30.613.750	18.133.750	40,77%
Zigue-zague	24.997.500	2.501	99,99%*
Quase ordenado	654.922	377.149	42,41%
Duplicados	16.633.993	6.300.346	62,12%
Invertido	49.995.000	0	100,00%**

* 99,99% (2.501 inversões restantes); ** 100,00% exato (zero inversões restantes). A diferença de arredondamento impede interpretação de equivalência entre os dois casos.

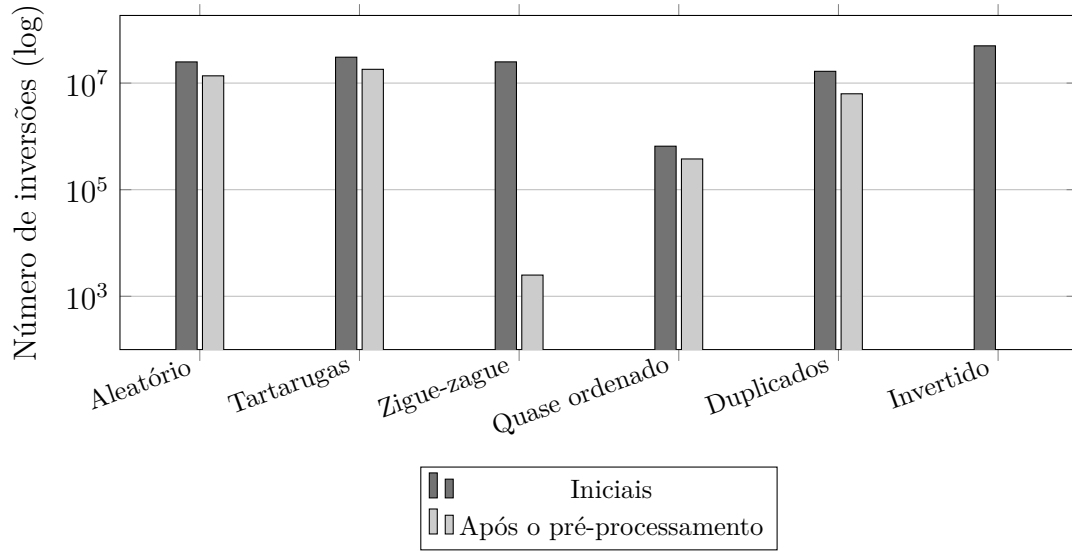


Figura 2. Número de inversões antes e após o pré-processamento simétrico para vetores de 10.000 elementos (escala logarítmica; no cenário Invertido o valor pós-pré é zero e não aparece na escala log).

Como o Algoritmo 1 executa a troca direta entre $a[i]$ e $a[n - 1 - i]$, um vetor estritamente invertido (onde $a[i] > a[n - 1 - i]$ para todo $i < n/2$) sofre uma reversão geométrica completa da sua sequência em $O(n)$ operações. A resolução do caso Invertido decorre da geometria da transformação do pré-processamento, configurando um caso ideal onde a topologia da desordem coincide perfeitamente com a simetria aplicada.

O comportamento observado no cenário *Zigue-zague* também pode ser interpretado à luz dessa propriedade estrutural do pré-processamento. Embora sua topologia não corresponda a uma inversão global como no caso *Invertido*, a alternância sistemática entre blocos com orientações distintas produz relações de desordem que são parcialmente capturadas pelas comparações entre posições simétricas. Dessa forma, a troca direta entre $a[i]$ e $a[n - 1 - i]$ corrige simultaneamente um grande número de relações fora de ordem, reduzindo as inversões de 24.997.500 para apenas 2.501, uma redução de aproximadamente 100% (99,99%).

O resultado evidencia que a eficácia do pré-processamento não se restringe ao caso de reversão completa: ela também se manifesta em topologias cuja estrutura apresenta forte alinhamento com a simetria utilizada pela transformação, ainda que de maneira parcial. Essa característica ajuda a explicar os ganhos particularmente elevados observados posteriormente nos algoritmos de Insertion Sort e Bubble Sort para essa entrada.

4.2. Impacto no Tempo de Execução e Custo do Pré-processamento

A Tabela 4 apresenta o tempo médio de execução e o ganho relativo obtido por cada algoritmo sob vetores de 10.000 elementos, evidenciando como a eficiência do pré-processamento é condicionada pela complexidade e adaptabilidade nativas de cada método. Os algoritmos quadráticos sensíveis à ordenação (como *Insertion Sort* e *Bubble Sort*) obtiveram os maiores benefícios práticos: no cenário *Zigue-*

zague, o pré-processamento reduziu o tempo do *Insertion Sort* de 5.535,18 μs para 16,44 μs (+99,7%) e do *Bubble Sort* de 19.147,42 μs para 17,67 μs (+99,9%), refletindo a eliminação drástica das inversões (de 24.997.500 para 2.501 inversões). Em contrapartida, métodos de complexidade assintótica $O(n \log n)$, como o *Merge Sort*, apresentaram resultados mistos (com degradação de até 17,8% e ganhos de até 14,8% em $n=10.000$; ganhos de até 18,5% em $n=10^6$, Tabela 6). Como a estrutura de divisão e conquista já garante alta eficiência nativa, o custo computacional da etapa prévia tende a suplantear eventuais ganhos em distribuições de baixa estrutura. Variações foram julgadas relevantes segundo o critério de não sobreposição dos ICs de 95% (Seção 3.3).

Tabela 4. Tempo médio por execução (em μs , média $\pm$ meia-largura do IC 95%) e ganho do pré-processamento para vetores de 10.000 elementos. Fonte: Criterion, seed 42. Critério de relevância: ICs de 95% não sobrepostos.

Algoritmo	Tipo	Puro	Com pré	Ganho
Merge Sort	Aleatório	429,35 $\pm$ 1,95	444,49 $\pm$ 5,94	-3,5%
Merge Sort	Tartarugas	125,17 $\pm$ 2,00	147,49 $\pm$ 4,30	-17,8%
Merge Sort	Zigue-zague	136,82 $\pm$ 1,30	116,63 $\pm$ 1,30	+14,8%
Merge Sort	Quase ordenado	135,04 $\pm$ 0,97	143,12 $\pm$ 1,31	-6,0%
Merge Sort	Duplicados	199,19 $\pm$ 1,70	201,32 $\pm$ 1,47	-1,1%
Merge Sort	Invertido	129,15 $\pm$ 2,41	112,21 $\pm$ 0,91	+13,1%
Quicksort	Aleatório	367,27 $\pm$ 2,03	382,50 $\pm$ 1,25	-4,1%
Quicksort	Tartarugas	197,67 $\pm$ 2,55	207,13 $\pm$ 2,41	-4,8%
Quicksort	Zigue-zague	540,28 $\pm$ 9,78	562,13 $\pm$ 8,28	-4,0%
Quicksort	Quase ordenado	306,46 $\pm$ 2,16	315,72 $\pm$ 2,55	-3,0%
Quicksort	Duplicados	44,92 $\pm$ 0,43	59,00 $\pm$ 0,47	-31,3%
Quicksort	Invertido	471,18 $\pm$ 3,96	644,59 $\pm$ 5,75	-36,8%
Insertion Sort	Aleatório	5506,86 $\pm$ 73,84	3181,76 $\pm$ 63,63	+42,2%
Insertion Sort	Tartarugas	6618,39 $\pm$ 79,22	4037,39 $\pm$ 76,08	+39,0%
Insertion Sort	Zigue-zague	5535,18 $\pm$ 57,97	16,44 $\pm$ 0,73	+99,7%
Insertion Sort	Quase ordenado	182,82 $\pm$ 3,93	130,58 $\pm$ 2,05	+28,6%
Insertion Sort	Duplicados	3662,06 $\pm$ 39,36	1488,72 $\pm$ 26,80	+59,3%
Insertion Sort	Invertido	10897,85 $\pm$ 160,40	13,04 $\pm$ 0,26	+99,9%
Bubble Sort	Aleatório	23676,19 $\pm$ 117,13	20530,09 $\pm$ 203,74	+13,3%
Bubble Sort	Tartarugas	19428,15 $\pm$ 133,43	14094,37 $\pm$ 94,62	+27,5%
Bubble Sort	Zigue-zague	19147,42 $\pm$ 117,96	17,67 $\pm$ 0,43	+99,9%
Bubble Sort	Quase ordenado	16466,39 $\pm$ 83,51	13943,78 $\pm$ 141,98	+15,3%
Bubble Sort	Duplicados	20376,82 $\pm$ 100,35	17055,89 $\pm$ 97,02	+16,3%
Bubble Sort	Invertido	21307,46 $\pm$ 255,90	10,28 $\pm$ 0,40	+100,0%
Selection Sort	Aleatório	21487,72 $\pm$ 137,81	21390,89 $\pm$ 106,85	+0,5%
Selection Sort	Tartarugas	21429,88 $\pm$ 171,35	20717,84 $\pm$ 122,11	+3,3%
Selection Sort	Zigue-zague	21382,51 $\pm$ 180,13	20975,92 $\pm$ 90,55	+1,9%
Selection Sort	Quase ordenado	21226,00 $\pm$ 116,05	21457,09 $\pm$ 279,79	-1,1%
Selection Sort	Duplicados	20626,01 $\pm$ 77,19	20788,64 $\pm$ 142,61	-0,8%
Selection Sort	Invertido	18262,67 $\pm$ 121,85	20515,49 $\pm$ 49,68	-12,3%

A viabilidade prática dessa abordagem fundamenta-se no baixo e altamente previsível custo computacional da etapa simétrica. Conforme demonstrado na

Tabela 5 e na Figura 3, o tempo de execução do pré-processamento apresenta um comportamento estritamente linear $O(n)$ em relação ao tamanho do vetor, com tempo unitário entre 1,18 e 1,36 ns por elemento, que decai monotonicamente com o crescimento de n e estabiliza em $\approx 1,18$ ns para 10^6 elementos, indicando ausência de gargalos de localidade de memória mesmo em escala. Assim, a varredura impõe um *overhead* irrisório — apenas $12,44 \mu s$ para 10.000 elementos —, amplamente compensado pela economia de milissegundos na ordenação adaptativa.

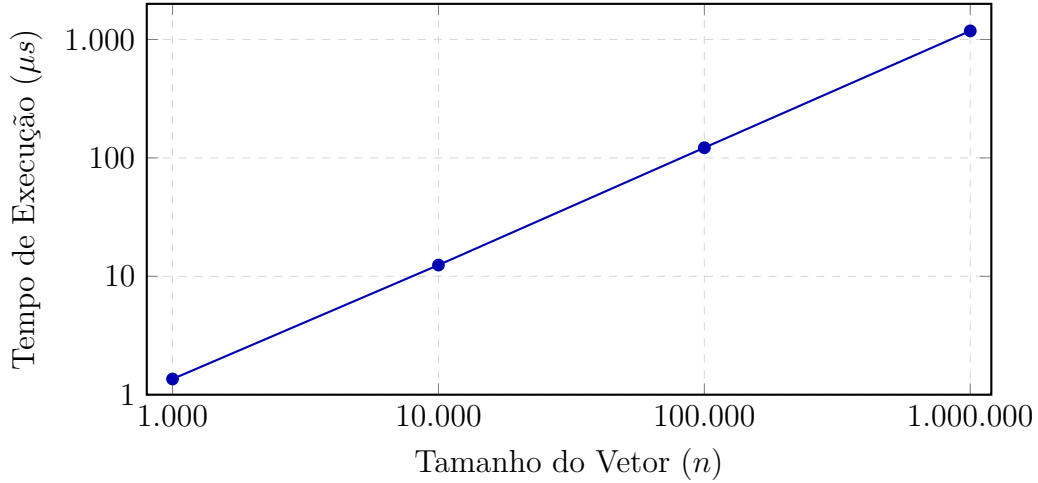


Figura 3. Tempo médio de execução do pré-processamento em função do tamanho do vetor (n), apresentado em escala log-log para evidenciar o comportamento linear $O(n)$. Valores médios sobre os seis tipos de entrada, obtidos pela ferramenta CLI com seed 42.

Tabela 5. Custo médio do pré-processamento simétrico em função do tamanho do vetor (média sobre os seis tipos; medido via ferramenta CLI, seed 42). O ponto $n=20.000$ foi incluído exclusivamente para caracterização da linearidade e não faz parte da matriz principal de tempos de ordenação.

Tamanho	Pré (μs)	ns/elemento
1.000	1,36	1,36
5.000	6,25	1,25
10.000	12,44	1,24
20.000	24,56	1,23
100.000	121,89	1,22
1.000.000	1.181,84	1,18

4.3. Tempo Total de Ordenação e Análise Individual dos Algoritmos

A avaliação do tempo total de execução revela uma dicotomia no impacto do pré-processamento: nos algoritmos adaptativos de classe quadrática, notadamente o *Insertion Sort* e o *Bubble Sort*, a drástica eliminação prévia das inversões resulta em ganhos substanciais de desempenho (variando entre 13% e 100% em $n=10.000$), justificados pela matemática do algoritmo — o modesto custo linear da varredura simétrica ($\approx 1,2$ ns/elemento) é vastamente amortizado pela supressão das operações

de trocas locais. Em contraste, os métodos de complexidade $O(n \log n)$ apresentam respostas qualitativamente distintas.

O *Merge Sort* exibe resultados mistos segundo o critério de não sobreposição de ICs. O ganho concentra-se nas topologias em que o pré-processamento elimina a desordem estrutural: *Invertido* (+13,1%) e *Zigue-zague* (+14,8%) em $n = 10.000$, com ganhos crescentes até +18,5% em $n = 10^6$ (Tabela 6). Em *Aleatório*, *Tartarugas* e *Quase ordenado*, porém, o *overhead* da etapa preliminar supera seus benefícios, resultando em degradação estatisticamente relevante.

O *Quicksort*, implementado com particionamento de três vias e mediana de três, apresenta degradação sistemática sob o pré-processamento. A perda é leve na maioria das topologias (−3% a −5% em *Aleatório*, *Tartarugas*, *Zigue-zague* e *Quase ordenado*), mas torna-se acentuada em *Duplicados* (−31,3%) e *Invertido* (−36,8%), sempre com ICs não sobrepostos. Esse comportamento é compatível com a natureza pouco adaptativa da implementação, cujo custo é fortemente influenciado pelas varreduras de particionamento, pouco afetadas pela redução das inversões.

Em *Invertido*, a transformação da entrada em uma sequência estritamente ordenada, com redução de 49.995.000 para 0 inversões, remove a estrutura explorada pela seleção de pivô por mediana de três, mas adiciona o custo das varreduras do pré-processamento. Em *Duplicados*, a degradação de −31,3% ocorre apesar da redução de 16.633.993 para 6.300.346 inversões (62,1%). A hipótese é que a reorganização perturba a distribuição de chaves iguais que favorecia o particionamento de três vias no vetor puro, cujo tempo em $n = 10.000$ foi de 44,92, μs . Como cada posição de *Duplicados* é gerada independentemente segundo $\mathcal{U}0,1,2$, esse agrupamento corresponde às próprias chaves iguais exploradas pelo particionamento DNF, e não a blocos contíguos pré-existentes.

1

Assim, para a classe $O(n \log n)$, a técnica não se mostra benéfica na implementação avaliada, com poucas exceções pontuais, como *Quase ordenado* em $n = 10^6$ (+11,1%, Tabela 6). Esses resultados delimitam a aplicabilidade do pré-processamento, indicando que a redução de inversões, isoladamente, não garante ganhos para algoritmos dessa classe.

O comportamento do *Selection Sort* confirma sua natureza não adaptativa: como o número de comparações é fixo ($n(n-1)/2$) independentemente da configuração inicial, a redução das inversões não se traduz em ganho algorítmico. De fato, nas medições o algoritmo permaneceu estatisticamente inalterado na maioria das topologias (variações de −1,1% a +3,3% com ICs sobrepostos, critério da Seção 3.3), com exceção de uma degradação consistente de aproximadamente 12% no cenário *Invertido* em $n = 10.000$ (18.262,67 $\pm$ 121,85 μs vs. 20.515,49 $\pm$ 49,68 μs ; ICs não sobrepostos), também reproduzida em $n = 100.000$ (−12,8%, 1.849,65 $\pm$ 9,39 ms vs. 2.086,10 $\pm$ 7,92 ms).

¹Os tempos e as contagens de inversões são medidas diretas (Tabelas 3 e 4); a atribuição causal a particionamento e predição de desvios constitui hipótese explicativa, não medição direta de *branch mispredictions*, indisponível no Windows (Seção 3.4).

Como o pré-processamento transforma a entrada invertida em sequência ordenada, a busca do mínimo em cada varredura passa a encontrar o menor elemento já na primeira posição candidata; o ganho esperado seria apenas evitar uma troca por iteração, o que não justifica a degradação observada. A divergência, portanto, parece decorrer de fatores microarquiteturais — especificamente do preditor de desvios (*branch predictor*) da CPU [Hennessy and Patterson 2019, Edelkamp and Weiß 2016].

Em vetores pré-estruturados, a condição central de atualização do índice mínimo torna-se sistematicamente falsa para sequência ordenada, gerando padrão de desvios distinto do observado na sequência invertida original. Trata-se de hipótese plausível, não de medição direta de *branch mispredictions* (indisponível no Windows, Seção 3.4). O relevante é que a magnitude dessas variações é limitada (no máximo $\sim 12\%$) e não configura tendência sistemática de ganho — consistente com a previsão teórica de que um algoritmo não adaptativo não explora a redução de inversões [Estivill-Castro and Wood 1992].

Ao acessar simultaneamente posições nas extremidades do vetor, o pré-processamento simétrico introduz dependências adicionais de controle e aumenta o número de instruções executadas por elemento. Em instâncias aleatórias, as condições internas de troca (`if A[i] > A[j]`) tornam-se difíceis de prever para o preditor de desvios [Edelkamp and Weiß 2016, Kaligosi and Sanders 2006]. Consequentemente, em cenários onde a redução de inversões é modesta, as penalidades arquiteturais e o custo extra de instruções superam os benefícios lógicos da reestruturação.

A Figura 4 sintetiza essa dinâmica: aproveitamento expressivo nos métodos quadráticos adaptativos, resposta mista do *Merge Sort*, degradação sistemática do *Quicksort* de três vias e neutralidade do *Selection Sort* não adaptativo (com a exceção do cenário Invertido). A Tabela 6 complementa a análise para $n=10^6$ nos métodos quasilineares.

Tabela 6. Tempo médio (ms) e ganho do pré-processamento para vetores de 1.000.000 elementos — apenas métodos quasilineares (quadráticos omitidos por inviabilidade temporal). Fonte: Criterion, 50 amostras, IC 95%.

Algoritmo	Tipo	Puro (ms)	Com pré (ms)	Ganho
Merge Sort	Aleatório	66,14 $\pm$ 0,75	67,12 $\pm$ 0,76	-1,5%
Merge Sort	Tartarugas	17,92 $\pm$ 0,30	20,03 $\pm$ 0,51	-11,8%
Merge Sort	Zigue-zague	19,39 $\pm$ 0,39	15,81 $\pm$ 0,26	+18,5%
Merge Sort	Quase ordenado	20,39 $\pm$ 0,39	21,34 $\pm$ 0,39	-4,7%
Merge Sort	Duplicados	25,85 $\pm$ 0,42	25,09 $\pm$ 0,26	+2,9%
Merge Sort	Invertido	18,46 $\pm$ 0,47	15,46 $\pm$ 0,19	+16,2%
Quicksort	Aleatório	53,83 $\pm$ 0,18	55,75 $\pm$ 0,26	-3,6%
Quicksort	Tartarugas	151,34 $\pm$ 0,86	157,07 $\pm$ 3,10	-3,8%
Quicksort	Zigue-zague	458,73 $\pm$ 2,59	506,44 $\pm$ 3,06	-10,4%
Quicksort	Quase ordenado	42,61 $\pm$ 0,27	37,88 $\pm$ 0,27	+11,1%
Quicksort	Duplicados	4,39 $\pm$ 0,02	5,70 $\pm$ 0,03	-29,9%
Quicksort	Invertido	416,59 $\pm$ 2,02	591,80 $\pm$ 4,62	-42,1%

Os valores de $n=10^6$ replicam o padrão de $n=10.000$: ganhos do Merge Sort restritos a Zigue-zague (+18,5%) e Invertido (+16,2%), degradação sistemática

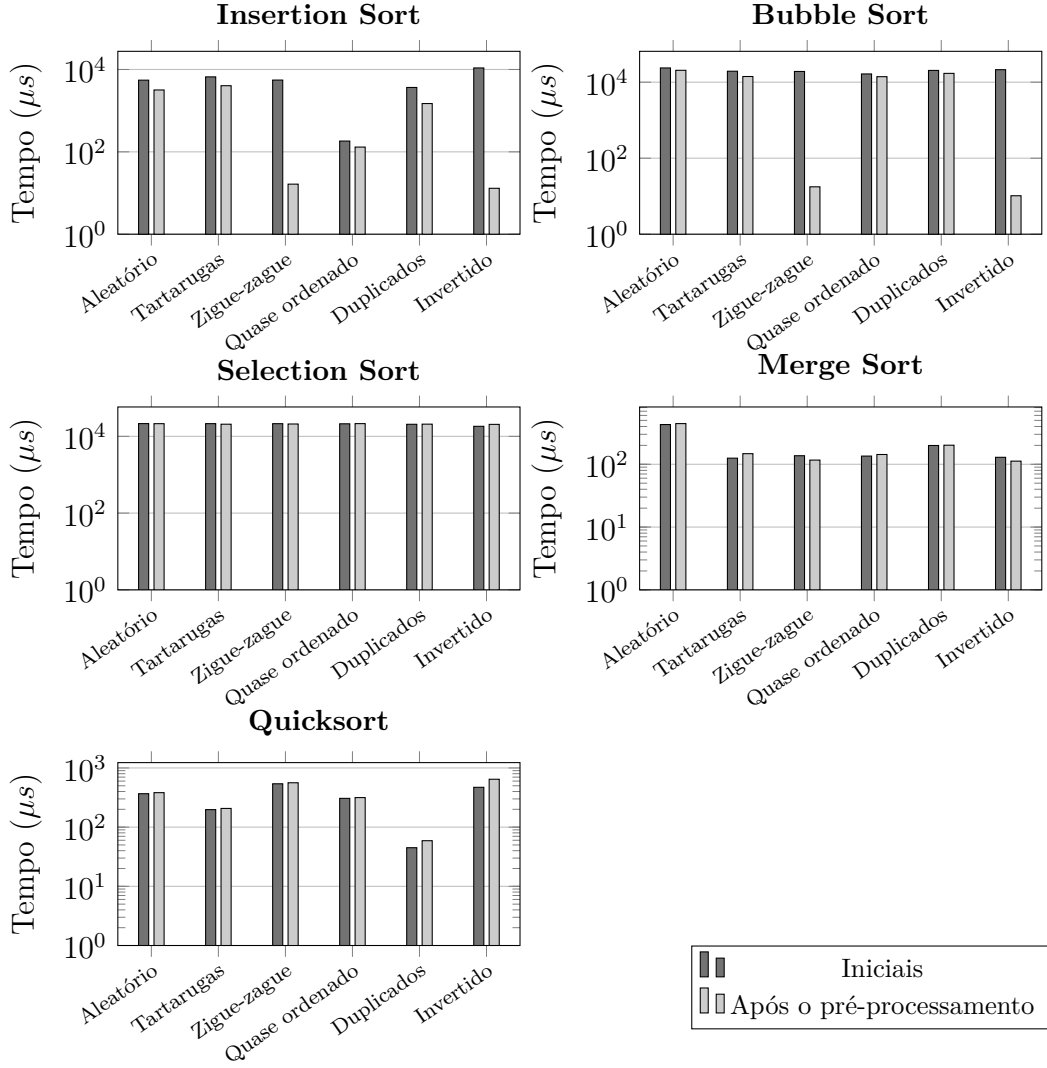


Figura 4. Comparação temporal entre a ordenação pura e com pré-processamento simétrico ($n = 10.000$), em escala logarítmica.

do Quicksort exceto em Quase ordenado (+11,1%), confirmando a delimitação de aplicabilidade aos métodos adaptativos.

5. Considerações Finais

Este trabalho apresentou e avaliou um pré-processamento simétrico de complexidade $O(n)$ destinado à redução do número de inversões em sequências de entrada para algoritmos de ordenação. Os resultados demonstraram que a abordagem pode proporcionar ganhos relevantes em algoritmos quadráticos adaptativos (Insertion Sort e Bubble Sort) quando aplicada a entradas com padrões estruturados de desordem ou em ordem inversa, com ganhos de 13% a 100% em $n=10.000$ e custos de pré-processamento de $\approx 1,2$ ns/elemento. Esse ganho é expressivo especialmente quando a redução do custo de ordenação supera o custo adicional introduzido pela etapa preliminar, satisfazendo a condição $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$.

Em relação ao algoritmo quadrático não adaptativo avaliado, o Selection

Sort confirmou a hipótese de neutralidade: variações de $-1,1\%$ a $+3,3\%$ com ICs sobrepostos na maioria das topologias, com exceção de uma degradação consistente de $\sim 12\%$ na entrada totalmente invertida (ICs de 95% não sobrepostos em $n=10.000$ e $n=100.000$), atribuída a efeitos de predição de desvios. Esse resultado, previsto teoricamente pela invariância do número de comparações ($n(n-1)/2$), delimita claramente que a redução de inversões não se converte em ganho para métodos não adaptativos.

Por outro lado, nos algoritmos de complexidade $O(n \log n)$ avaliados, a aplicação do método revelou limitações importantes. Com a implementação de Quicksort utilizada (particionamento de três vias e mediana de três), o pré-processamento causou degradação sistemática de desempenho — de 3% a 5% na maioria das topologias e de 31% a 42% em *Duplicados* e *Invertido* (ICs não sobrepostos) — pois o custo desse método é dominado pelo número de varreduras de particionamento, pouco alterado pela redução de inversões.

No Merge Sort os efeitos foram mistos, com ganhos restritos às topologias estruturalmente mais afetadas pela varredura simétrica (Zigue-zague $+14,8\%$ em $n=10.000$ e $+18,5\%$ em $n=10^6$; Invertido $+13,1\%$ e $+16,2\%$, respectivamente) e degradação nas demais. Tal constatação indica que a viabilidade prática da técnica deve ser estritamente condicionada a algoritmos quadráticos adaptativos e a entradas com alta desordem estrutural, não se estendendo a métodos quasilineares de forma geral e, em particular, não beneficiando algoritmos não adaptativos como o Selection Sort.

Como trabalhos futuros, recomenda-se (i) derivação teórica fechada da redução esperada de inversões sob entrada aleatória no arcabouço de Hwang et al. [Hwang et al. 2000]; (ii) realização de testes em ambiente Linux com a infraestrutura `perf_event`, viabilizando a medição direta de *cache misses* e *branch mispredictions* para validar as hipóteses microarquiteturais aqui qualificadas como plausíveis; e (iii) exploração de particionamentos alternativos e técnicas adaptativas recentes como *BlockQuicksort* [Edelkamp and Weiß 2016] e *Powersort/Timsort* [Auger et al. 2018] como base comparativa.

Referências

- Auger, N., Jugé, V., Nicaud, C., and Pivoteau, C. (2018). On the worst-case complexity of timsort. In *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, volume 112 of *LIPIcs*, pages 4:1–4:13.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, 4th edition.
- Edelkamp, S. and Weiß, A. (2016). Blockquicksort: Avoiding branch mispredictions in quicksort. In *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, volume 57 of *LIPIcs*, pages 38:1–38:16.
- Estivill-Castro, V. and Wood, D. (1992). A survey of adaptive sorting algorithms. *ACM Computing Surveys*, 24(4):441–476.
- Gil, A. C. (2019). *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda.

- Hennessy, J. L. and Patterson, D. A. (2019). *Computer architecture: a quantitative approach*. Elsevier, Cambridge, Massachusetts, sixth edition edition. OCLC: 1196175914.
- Hwang, H.-K., Yang, B.-Y., and Yeh, Y.-N. (2000). Presorting algorithms: An average-case point of view. *Theoretical Computer Science*, 242(1-2):29–40.
- Kaligosi, K. and Sanders, P. (2006). How branch mispredictions affect quicksort. *Journal of Experimental Algorithmics*, 11. Também apresentado em ESA 2006.
- Knuth, D. E. (1973). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA.
- LaMarca, A. and Ladner, R. E. (1999). The Influence of Caches on the Performance of Sorting. *Journal of Algorithms*, 31(1):66–104.
- Mannila, H. (1985). Measures of presortedness and optimal sorting algorithms. *IEEE Transactions on Computers*, C-34(4):318–325. Versão estendida publicada originalmente como resumo no ICALP 1984 (LNCS 172, pp. 324–336).
- Nascimento, S. d. P. (2026). Presorting algorithm. Disponível em: <https://github.com/Zidan-09/presorting-algorithm>. Repositório de código-fonte e dados experimentais.
- Sampieri, R. H., Collado, C. F., and Lucio, M. d. P. B. (2021). *Metodologia de pesquisa*. Penso.
- Sedgewick, R. and Wayne, K. D. (2011). *Algorithms*. Addison-Wesley, Upper Saddle River, 4th edition.
- Ziviani, N. (2021). *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning.